Topic #:-

Red-Black Tree

=> it is a Self-balancing BST.

=> it was created in 1972 by Rudolf Bayer who termed them symmetric binary B-trees.

=> it is binary search tree with one extra bit of storage per node, its color which can be either Red or black.

=> By constraining the node colors on any simple path from the root to leaf, red black trees ensure that no such path is more than twice as long as any other, so that tree is approximately balanced.

=> Each node of the tree contains the attributes:

- Color
- key
- left
- Right & P.

- Properties of Red-Black tree:-

Root property:

- The root is black.

Red/black property:

- Every node is either red or black.

Leaf property:

- Every leaf (NIL) is black.

Red property:

- If a node is red, then both its children are black.

Depth property:

- For each node, all simple paths from the node to descendent leaves contain the same number of black nodes.

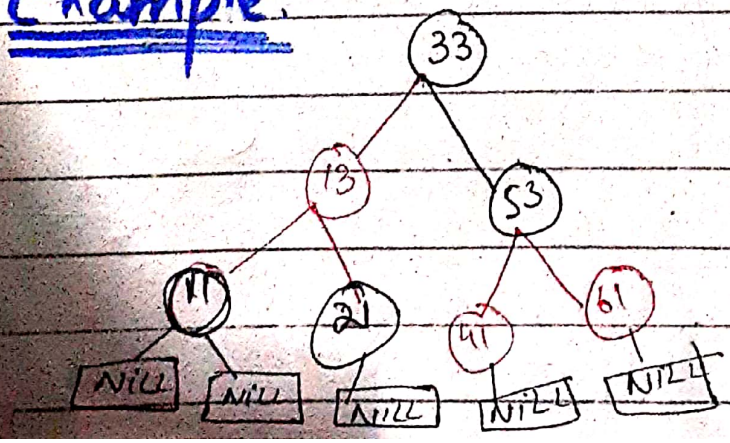
- Maximum-Rotations:

- Two + recoloring

- Time Complexity:

- O(log n)

Example:



Q: How the red-black tree maintains the property of self-balancing?

=> The red-black color is meant for balancing the tree.

The limitations are put on the node colors ensure that any simple path from the root to a leaf is not more than twice as long as any other such path.

=> it helps in maintaining the self-balancing property of the red-black trees.

Operations on Red-black tree:

- Various operations can be performed on red black tree.
  - =>Insertion
  - =>Deletion
  - =>Recolor
  - =>Rotation

Note: Every red-black tree is BST but every BST may not be a RB Tree.

Q: Why newly inserted nodes are always red in a red-black tree?

- Because inserting a red node does not violate the depth property of a red black tree.
- If we attach a red node to a red node, then the rule is violated but is easier to fix than the problem introduced by violating the depth property.

Lemma:

A red-black tree with n internal nodes has height at most $2\lg(n+1)$.

Q#: What is the largest possible no of internal nodes in a red-black tree with black-height k? what is the smallest possible number?

Insertion in Red-black Tree

Rules:

i) if tree is empty, Create newnode as root node with color black.

ii) if tree is not empty, create new node as leaf node with color Red.

iii) if parent of new Node is black then exit

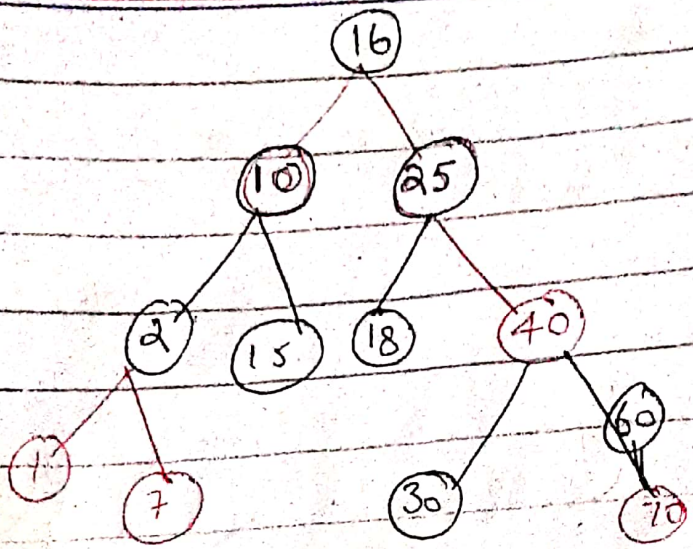
iv) if parent of new node is Red then check the color of parent's uncle/Sibling of new node.

a) if color is black or null then do suitable rotation & recolor

b) if color is red, then recolor and also check if parent's parent of new node is not root node then recolor it and recheck.

Example:

10, 18, 7, 15, 16, 30, 25.
40, 60, 2, 1, 70



Time complexity:

1 - insert : O(log n) ← maximum height of
2 - Color red : O(1)
3 - Fix violations :
constant # of :
a) Recolor : O(1)
b) Rotation : O(1)

Algorithm:-

RB-Insert (T, z)

y = T.nil ——— 1
x = T.root ——— 1
While x ≠ T.nil ⟶ (log n)
y = x ——— 1
if z.key < x.key ——— 1
x = x.left
else
x = x.right
z.p = y ——— 1

if $y == T.nil$ ——— (1)
$T.root = z$
else if $z.key < y.key$
$y.left = z$
else
$y.right = z$
$z.left = T.nil$ ——— (1)
$z.right = T.nil$ ——— (1)
$z.color = Red$ ——— (1)
RB-Insert-Fixup(T, z)

RB-Insert-Fixup(T, z)
While $z.p.color == Red$ — O(log n)
if $z.p == z.p.p.left$
$y = z.p.p.right$
if $y.color == Red$ // check uncle color
$z.p.color = Red$
$y.color = Black$
$z.p.p.color = Red$
$z = z.p.p$
case 1
else if $z == z.p.right$
$z = z.p$
case 2
Left-Rotate(T, z)
$z.p.color = Black$
$z.p.p.color = Red$
Right-Rotate(T, z.p.p)
case 3

else (Same as "then clause" with "right" and "left" exchanged)

T.root.color = Black

Left-Rotate (T, x)

y = x.right ——— 1

x.right = y.left

if y.left ≠ T.nil ——— 1

y.left.p = x

y.p = x.p

if x.p == T.nil ——— 1

T.root = y

else if x == x.p.left

x.p.left = y

else x.p.right = y

y.left = x

x.p = y ——— 1

Note:

Case 1: Uncle is red.

Case 2: Uncle is Black

- node is left child of right
- node is right child of left

Case 3: Uncle is Black

- node is left child of left child
- node is right child of right child

Q#: Let h be the height of some red-black tree and x be a leaf of the red-black tree. How many nodes can be on a path from the root to x?

h - nodes

$\frac{h}{2}$ - red nodes at most

$\frac{h}{2}$ - black nodes at least

$2^{\lfloor h/2 \rfloor} - 1$ at least internal nodes

4-Scenarios

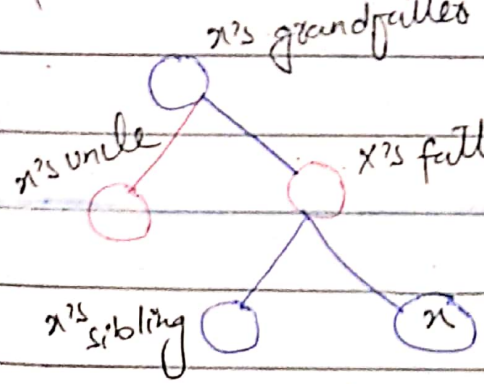
z = root -> color black

z.uncle = red -> recolor

z.uncle = black (triangle) -> rotate z.parent

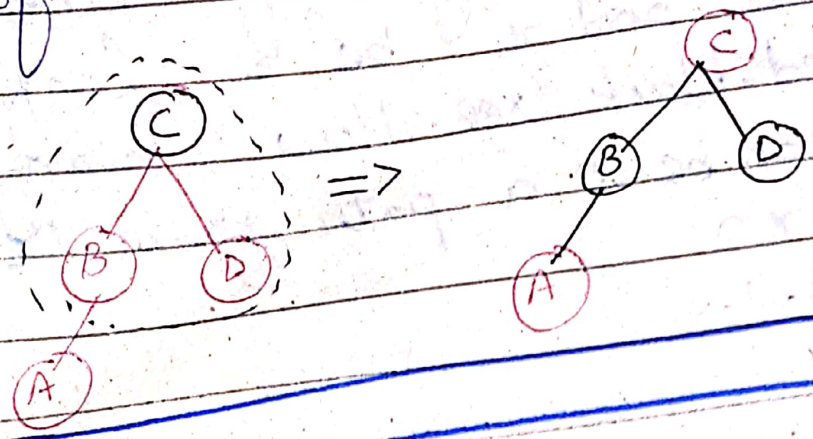
z.uncle = black (line) -> rotate z.grandparent & recolor

Representation:



Note:

Recoloring: Recolor whenever the sibling of a red node's red parent is red.



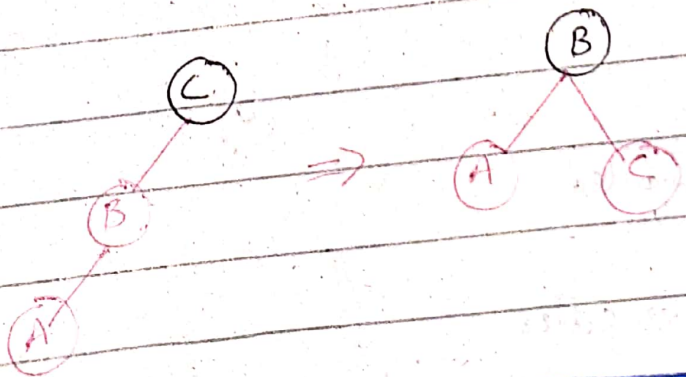
Note:

Restructuring: Restructuring whenever the red child's red parent's sibling is black or null.

There are four cases:

- Left
- Right
- RL
- LR

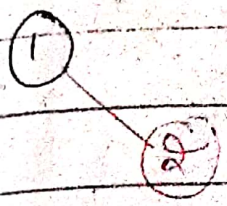
=> When restructure, the root of the restructured subtree is colored black and its children are colored red.



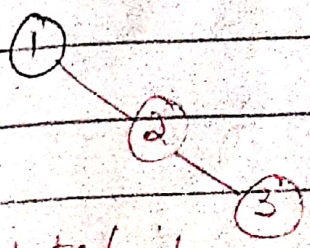
Create Red-black tree of the following data : 1, 2, 3, 4, 5, 6, 7, 8, 9

insert 1:

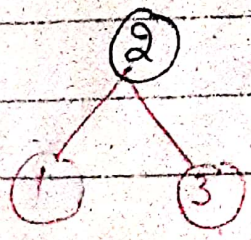
insert 2:



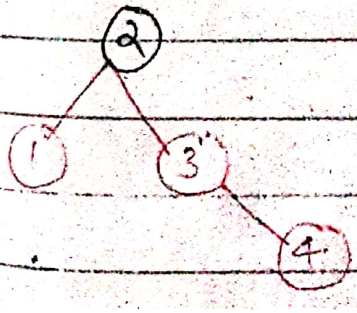
insert 3:



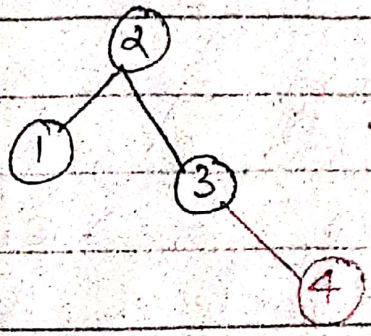
Rotate left



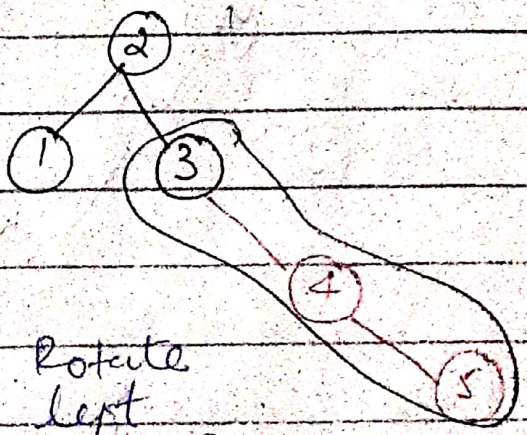
insert 4:



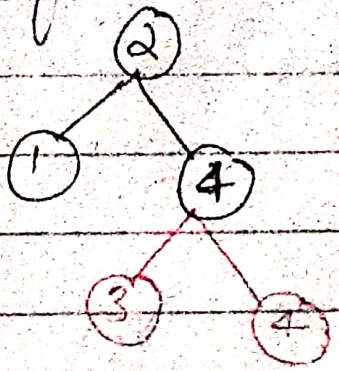
After recolor



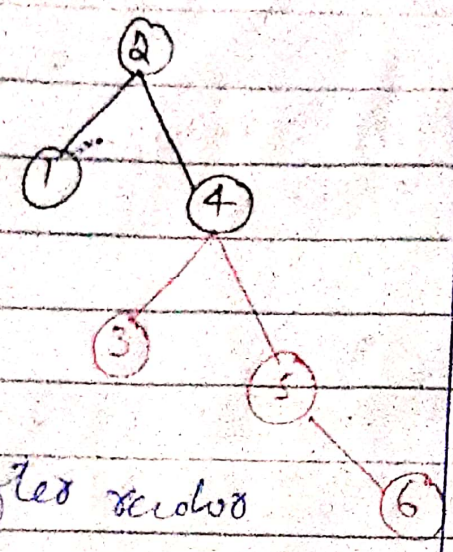
insert 5:



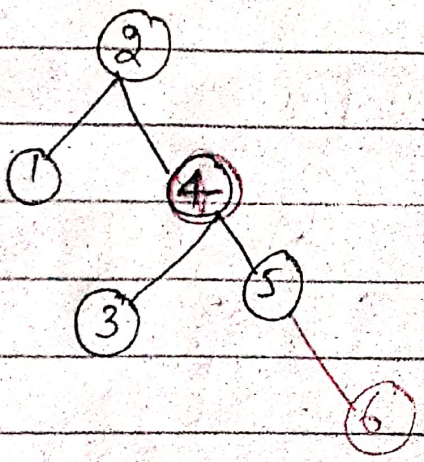
Rotate left



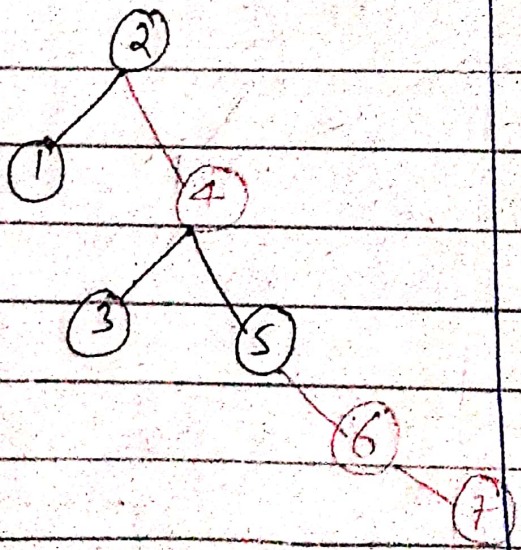
insert 6:



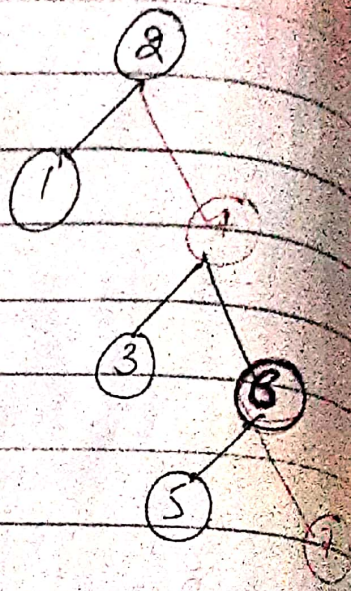
After recolor



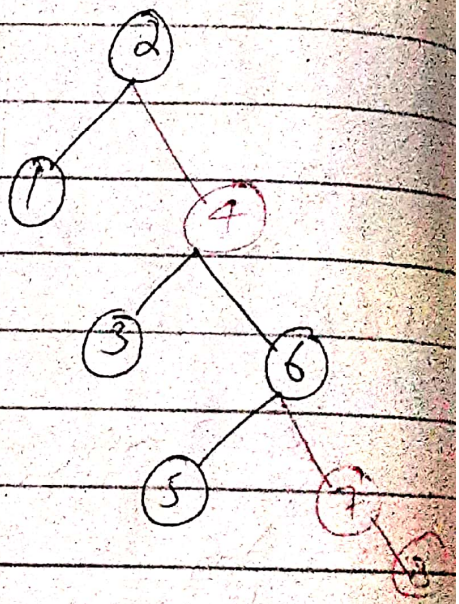
insert 7:



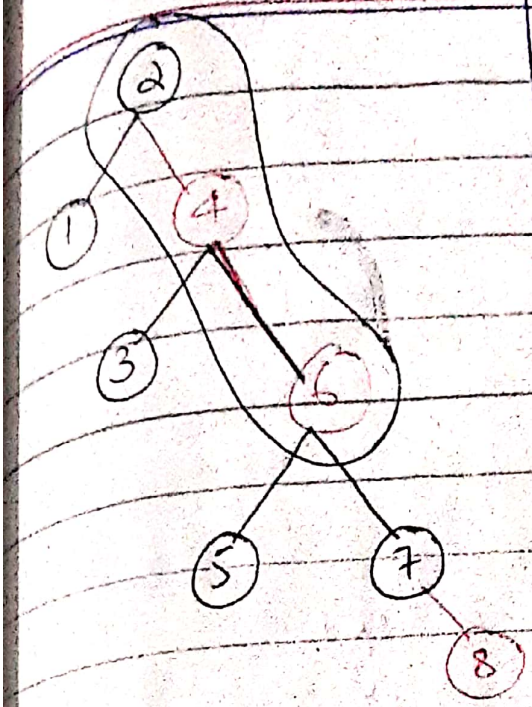
After rotate



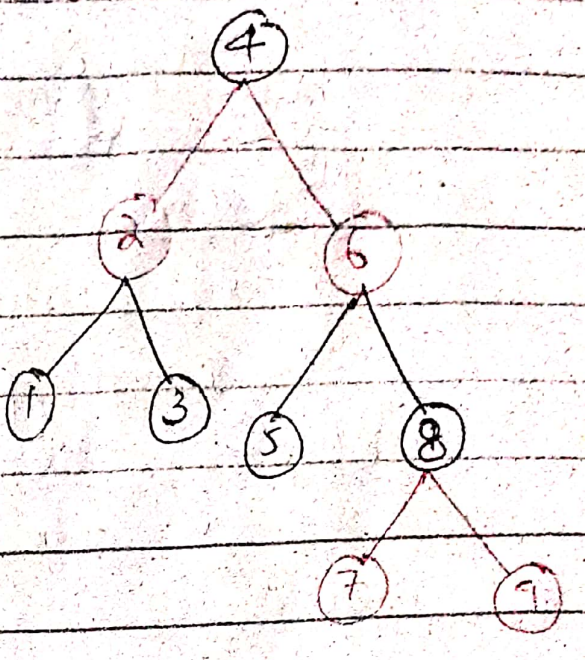
insert 8:

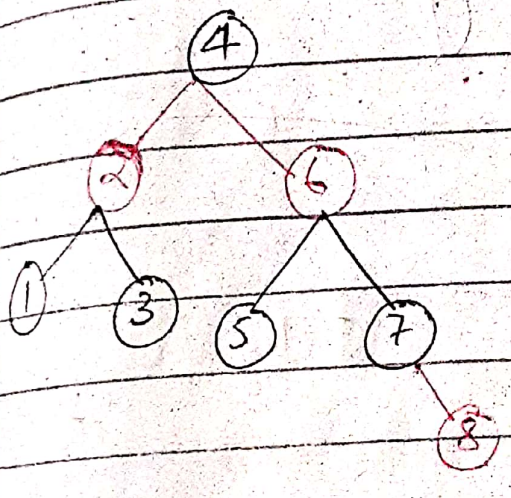


After recolor & rotate

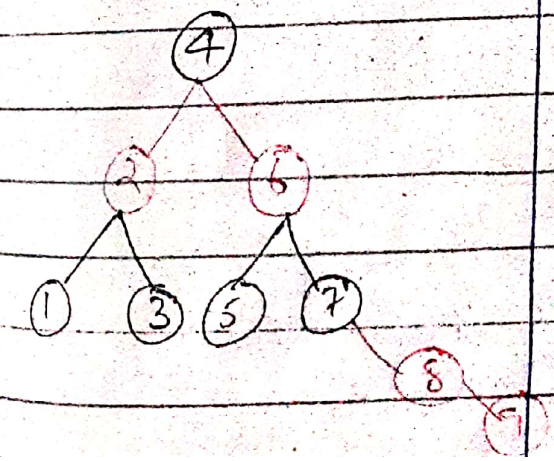


Rotate left





insert 9:



Right-Rotate (T, x)

y = x.left ——— 1

x.left = y.right ——— 1

y.p = x.p

if x.p == T.nill ——— 1

T.root = y

else if x = x.p.left

x.p.left = y

else

x.p.right = y

y.right = x ——— 1

x.p = y ——— 1

O(1)

Deletion in Red-black Tree

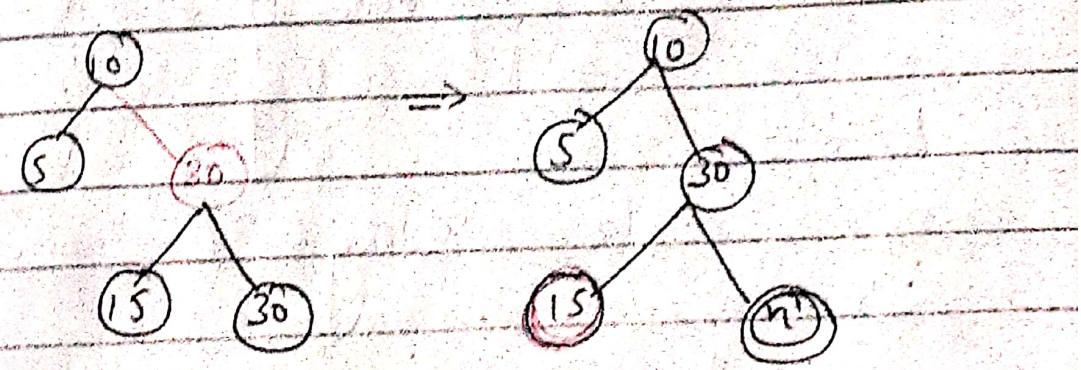
Step 1: Program BST deletion.

Case I: Node is red simply delete it.

Case II: if root is DB, just remove DB.

Case III

if DB's sibling is black and both its children are black



=> Remove DB

=> Add black to its parent

=> if p is red it becomes black

=> if p is black it becomes double black.

-> Make Sibling red.
-> if still DB exist, apply other cases.

Case 4:

=> if DB's Sibling is red
- Swap color of parent and its Sibling
- Rotate parent in DB direction.

Case 5:

=> DB sibling is black & left child is red and right child is black
- Swap color of DBs sibling and sibling left child
- Rotate Sibling in opposite direction to DB
- Apply case 6

Case 6:

=> DB sibling is black, left child of sibling is black and right child of sibling is red
- Swap color of parent & sibling

o Potale present in DB allocation.
remove DB
o change colon of red child
to black.

Algorithm:-

RB - Delete (T, z)
y - original_color = y.color ——— 1
if z.left == T.nil ——— 1
x = z.right
RB-Transplant (T, z, z.right)
else if z.right == T.nil
x = z.left
RB-Transplant (T, z, z.left)
else
y = Tree_Minimum (T.right) ——— 1
y-original_color = y.color
x = y.right
if y.p == z ——— 1
x.p = y
else
RB-Transplant (T, y, y.right)
y.right = z.right
y.right.p = y

RB-Transplant (T, z, y)

y.left = z.left — 1

y.left.p = y — 1

y.color = z.color — 1

if y-original-color == Black

RB-Delete-Fixup(T, x) O(lg n)

RB-Delete-Fixup (T, x)

while x ≠ T.root and x.color == Bl O(lg n)

if x == x.p.left — 1

w = x.p.right

if w.color == Red

w.color = Black

w.p.color = Red

Left-Rotate (T, x.p)

w = x.p.right

if w.left.color == Black, w.right.color

w.color = Red

x = x.p

else if w.right.color == Bl

w.left.color = Black

w.color = Red

Right-Rotate(T, x.p)

w = x.p.right

w.color = x.p.color

x.P : color = Black
x.right : color = Black
Left Rotate (T, x.P) — O(1)
x = T.root
else (same as "right" & Left "change")
x.color = Black

Time-Complexity

	Average	Worst case
Space	O(n)	O(n)
Search	O(log n)	O(log n)
insert	O(log n)	O(log n)
Delete	O(log n)	O(log n)

Applications of red black tree:

- To implement finite maps
- In Linux kernel
- To implement standard template libraries (STL) in C++: multiset, map, multimap

AVL VS Red-Black tree

AVL	Red-Black tree
o AVL is strictly height balanced	o Red-black is height balanced
o More rotations are required	o Less rotations are required
o Searching is faster	o Searching is not faster
o Slower insertion and removal	o Faster insertion & removal
o Each node has a balance factor	o No balance factor in RB tree.

Note: AVL is a subset of Red-black tree.